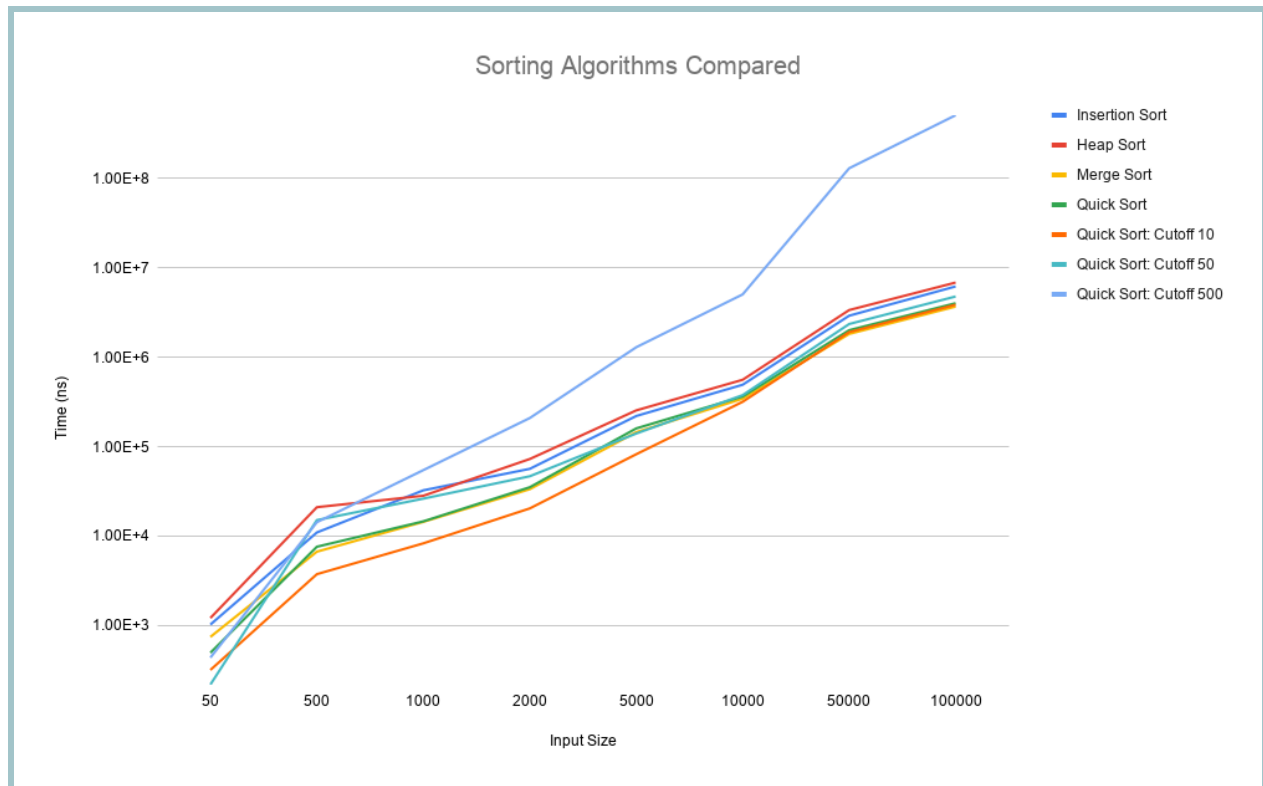




These tests did not produce any unexpected or erroneous data. I ran tests on 8 different input sizes and 3 different ranges, but there was little to no variation in sorting times on arrays with varying data ranges. The graph above displays the results from arrays with range 1000, and the attached csv file will include the rest of the testing results.

Insertion sort took far more time to complete than any of the other algorithms, due to its $O(n^2)$ time complexity. However, it actually performed relatively well at the smallest input sizes. This pattern demonstrates that insertion sort remains useful for small-scale sorting tasks.

Merge sort and heap sort times were fairly similar. At low input sizes, heap sort was faster, but merge sort performed much better at larger input sizes. I believe this is because merge sort has a smaller constant performance overhead that becomes evident when comparing the two algorithms. However, heap sort may be faster at smaller input sizes because merge sort has to allocate memory, which can be a slow operation that becomes noticeable at smaller input sizes.

The quick sort variations produced the fastest times. Of the quick sort variations, the optimal cutoff value was consistently 50. The 500 cutoff was the slowest by a substantial margin, and the 10 cutoff was extremely close in speed to unmodified quick sort. This makes sense to me, as at the input sizes I tested, a cutoff value of 500 would slow down the algorithm substantially due to a much higher usage of insertion sort. A cutoff of 50 seems to be the overall sweet spot for this set of input sizes.